Lecture 1c

VAE_Convolution_CELEBA_5.ipynb

What Is a Convolution?

🧠 The Basic Idea

> A **convolution** combines two functions into a third one that expresses how one is "shaped" by the other.

What Is a Convolution?

🧠 The Basic Idea

> A **convolution** combines two functions into a third one that expresses how one is "shaped" by the other.

📐 Mathematical Definition (1D Continuous)

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) \cdot g(t - \tau)\, d\tau$$

What Is a Convolution?

🧠 The Basic Idea

> A **convolution** combines two functions into a third one that expresses how one is "shaped" by the other.

📐 Mathematical Definition (1D Continuous)

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) \cdot g(t - \tau)\, d\tau$$

🔢 Discrete 1D Version (for sequences)

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k] \cdot g[n - k]$$

What Is a Convolution?

🧠 The Basic Idea

> A **convolution** combines two functions into a third one that expresses how one is "shaped" by the other.

📐 Mathematical Definition (1D Continuous)

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) \cdot g(t - \tau)\, d\tau$$

🔢 Discrete 1D Version (for sequences)

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k] \cdot g[n - k]$$

🧩 Intuitive Interpretation:

- Think of one function **sliding** across another.
- At each step, compute a **dot product** between overlapping values.
- The result is a **new sequence** that captures where patterns in the filter align with the input.

2D Convolution on Images

🔢 Mathematical Formulation (Discrete 2D)

Let I be the input image and K be the convolutional kernel (filter), then the 2D convolution at position (i, j) is defined as:

$$S(i,j) = \sum_m \sum_n K(m,n) \cdot I(i-m, j-n)$$

Or in practice (assuming kernel is centered and flipped):

$$S(i,j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K(u,v) \cdot I(i+u, j+v)$$

Where:

- I: input image matrix
- K: convolution kernel (e.g., 3×3 edge detector)
- $S(i,j)$: output at location (i,j)

🧭 Intuitive Description:

- The kernel slides **spatially** across the image
- At each position:
  - Compute the **elementwise product** between the kernel and the overlapping image patch
  - Sum the result → this becomes the output at that pixel
- This process **highlights** structures like:
  - Horizontal/vertical edges
  - Corners
  - Texture variations

Conv2d

CLASS torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

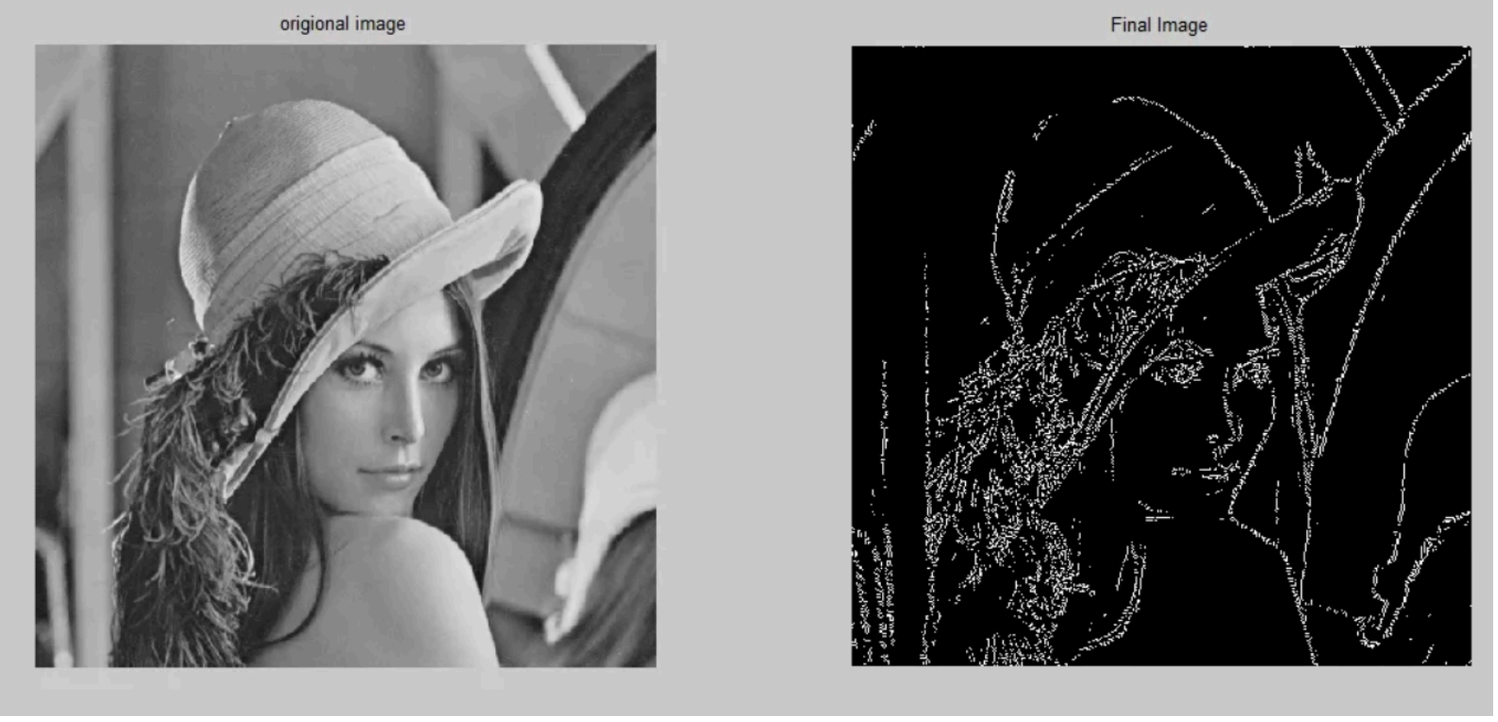
Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

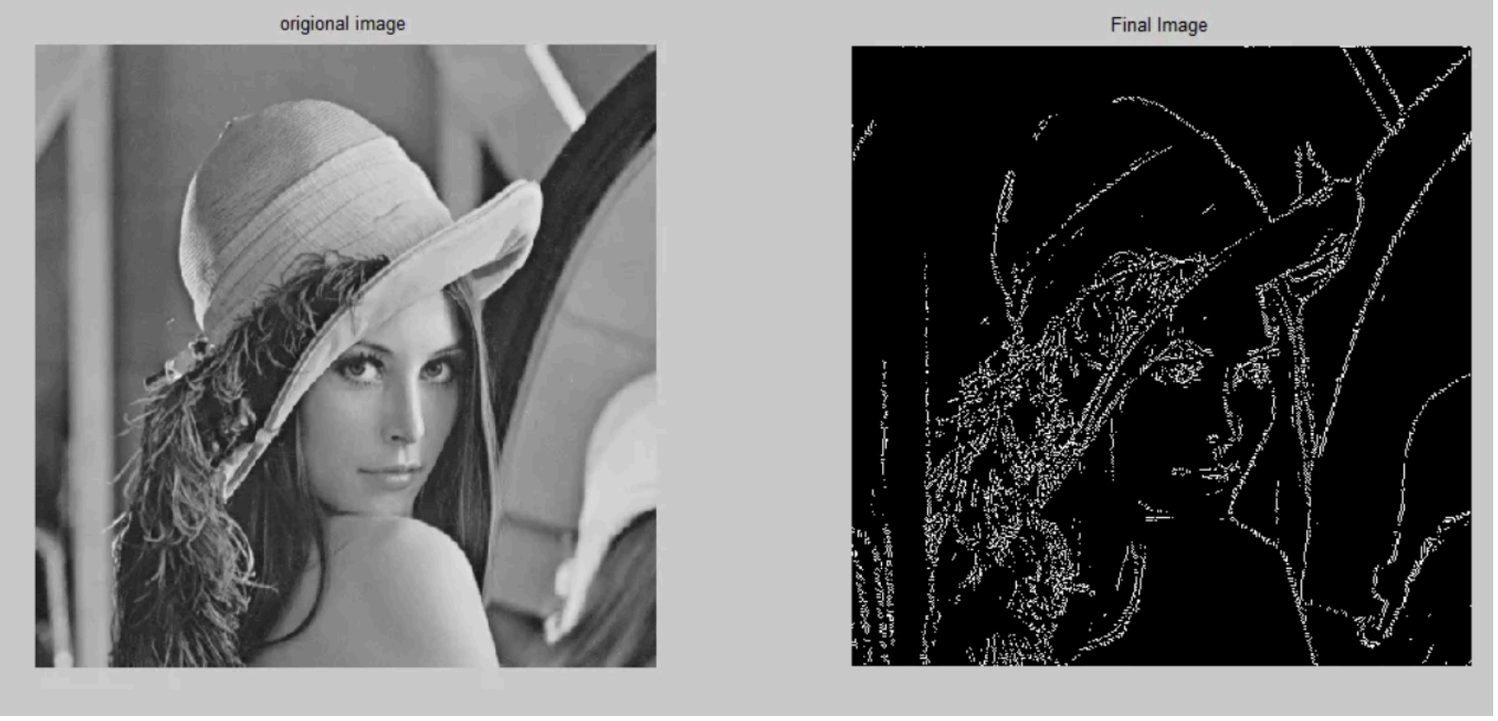
where $\star$ is the valid 2D cross-correlation operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

PyTorch



🔍 What is the Sobel Filter?

The **Sobel filter** is a pair of 3×3 kernels used to detect **edges** in images — specifically, **changes in intensity** (gradients) in the **horizontal and vertical directions**.



- **Horizontal Edge Detector (detects vertical gradients)**

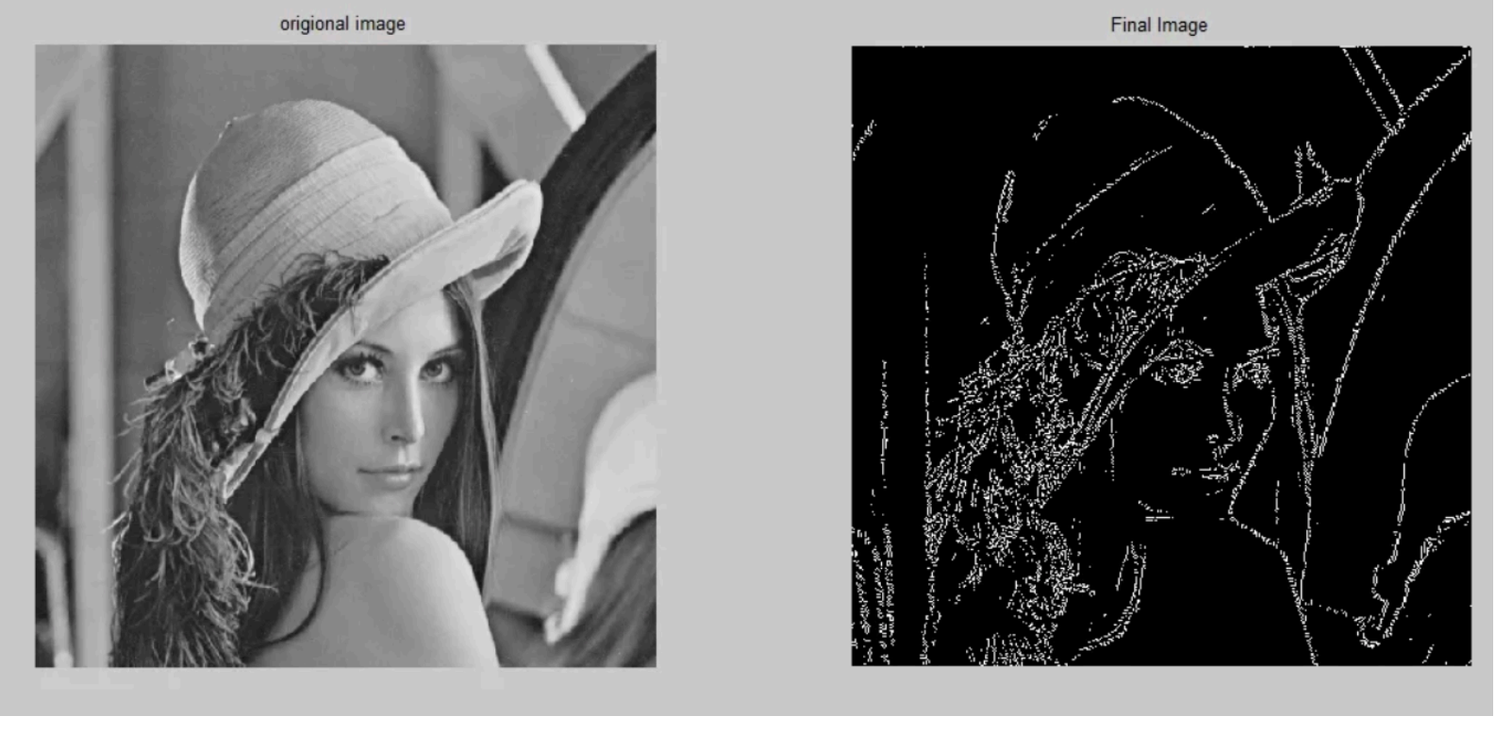
$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

- **Vertical Edge Detector (detects horizontal gradients)**

$$G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

🔍 What is the Sobel Filter?

The **Sobel filter** is a pair of 3×3 kernels used to detect **edges** in images — specifically, **changes in intensity** (gradients) in the **horizontal and vertical directions**.



- **Horizontal Edge Detector (detects vertical gradients)**

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

- **Vertical Edge Detector (detects horizontal gradients)**

$$G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

🔍 What is the Sobel Filter?

The **Sobel filter** is a pair of 3×3 kernels used to detect **edges** in images — specifically, **changes in intensity** (gradients) in the **horizontal and vertical directions**.

🧠 Intuition

- These kernels **emphasize intensity changes** in a particular direction.
- They do this by applying **larger weights** to pixels in the direction of the gradient, and subtracting pixels on the opposite side.
- For example, G_x will respond strongly to **vertical edges** (i.e., sudden horizontal changes in pixel values), such as where a light area meets a dark one from left to right.

What Are Filters and Feature Maps in CNNs?

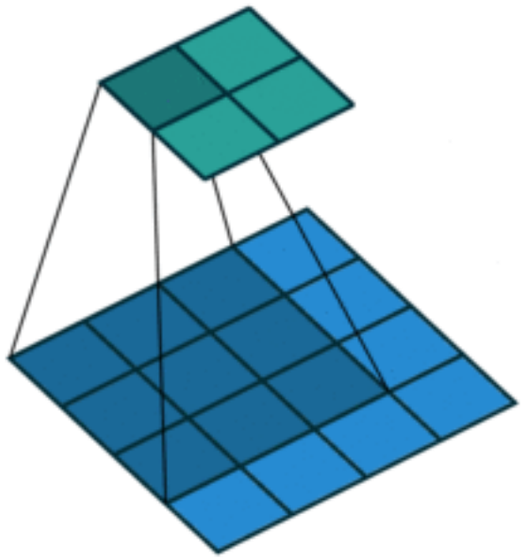
🔷 What Is a Filter?

- A **filter** (or kernel) is a small weight matrix — typically 3×3 or 5×5.
- Filters slide over the image and perform **convolutions** (as shown in previous slides).
- In CNNs, **filter weights are learned** during training — they are not hand-designed like Sobel.

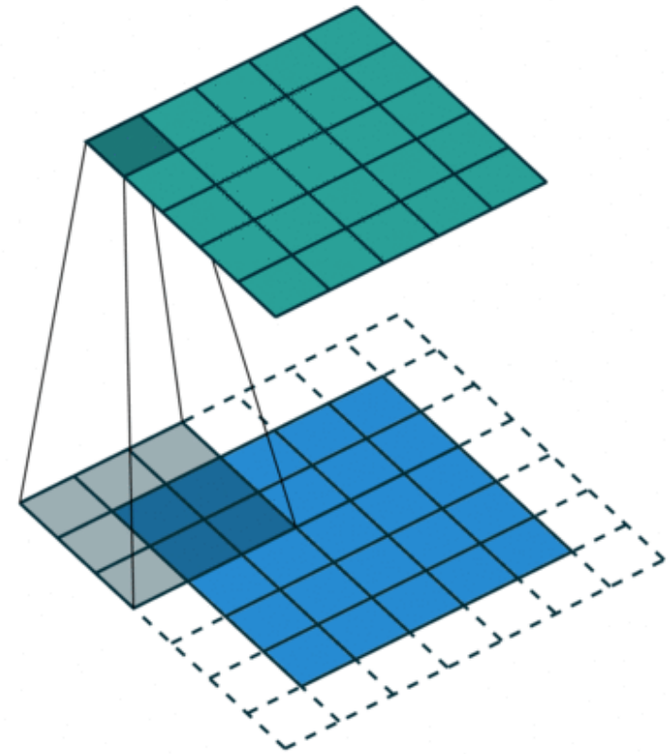
🔷 What Is a Feature Map?

- The **feature map** is the output of applying a filter to the input (image or previous feature map).
- It highlights **where in the image the filter "activates"** — i.e., where the learned pattern is present.

padding = 0, stride = 1



padding = 1, stride = 1

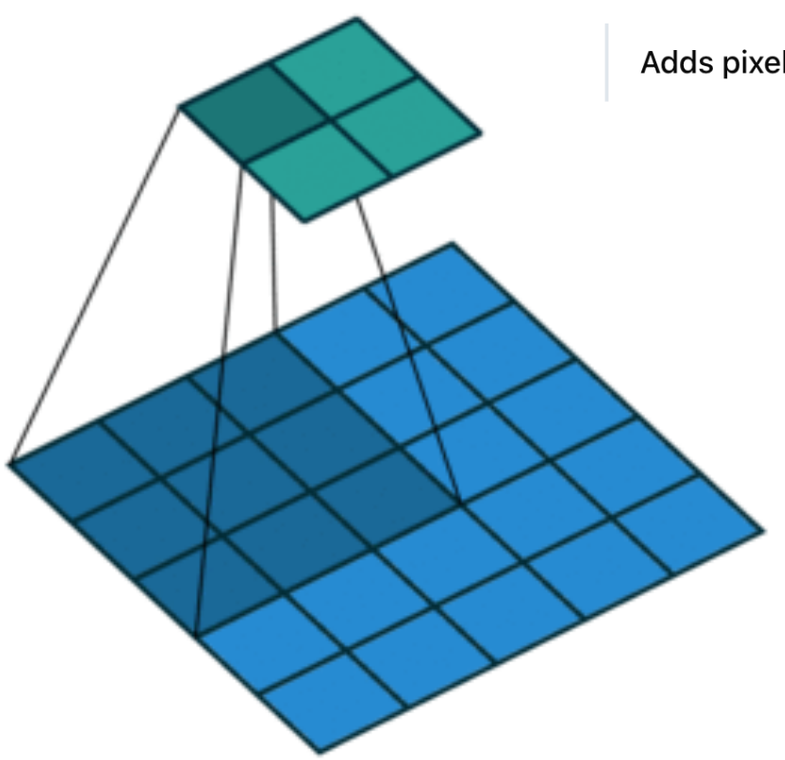


🔹 2. Stride

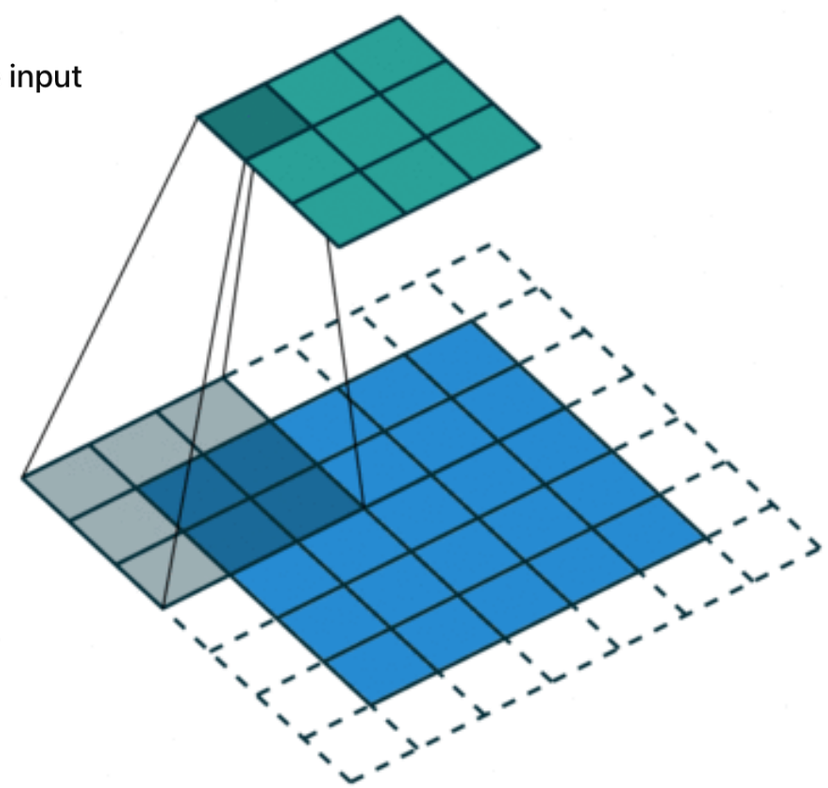
> The number of pixels the filter moves each step

https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html

padding = 0, stride = 2



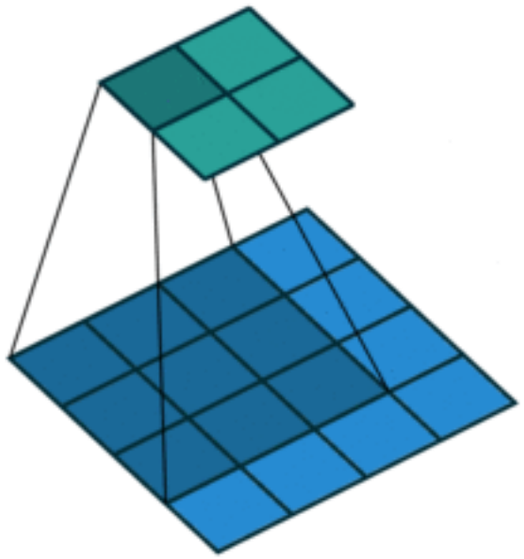
padding = 1, stride = 2



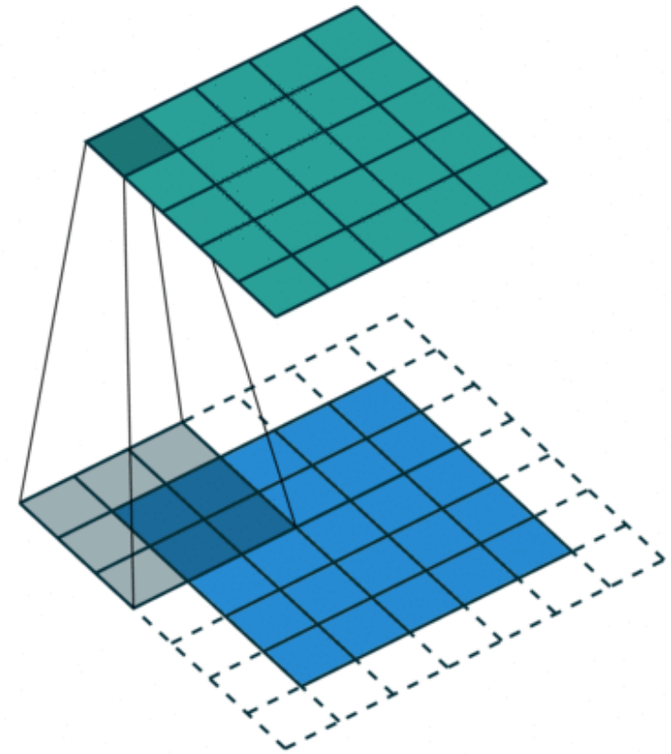
🔹 1. Padding

> Adds pixels (typically zeros) around the border of the input

padding = 0, stride = 1



padding = 1, stride = 1

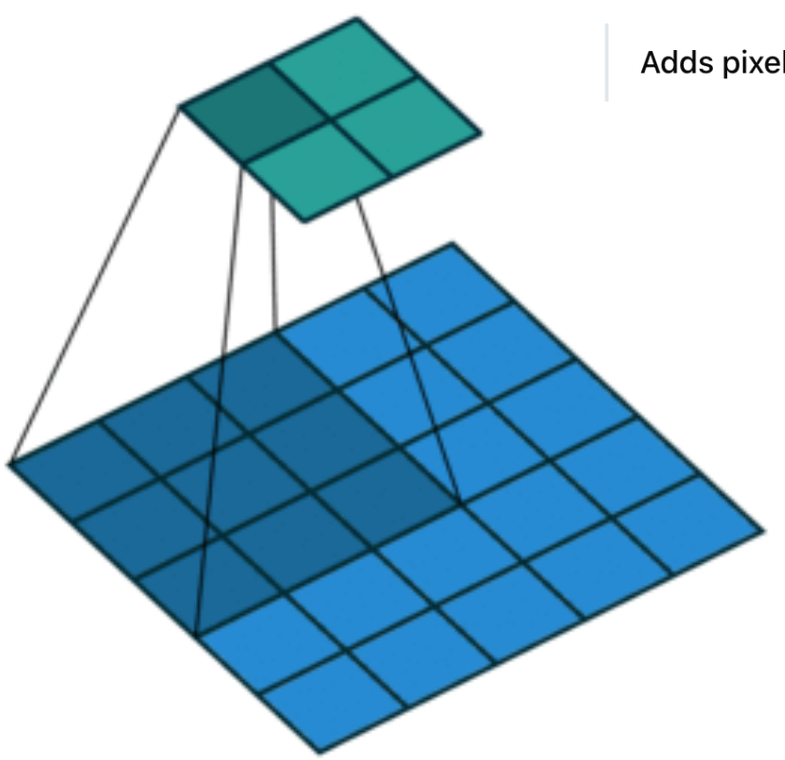


🔹 2. Stride

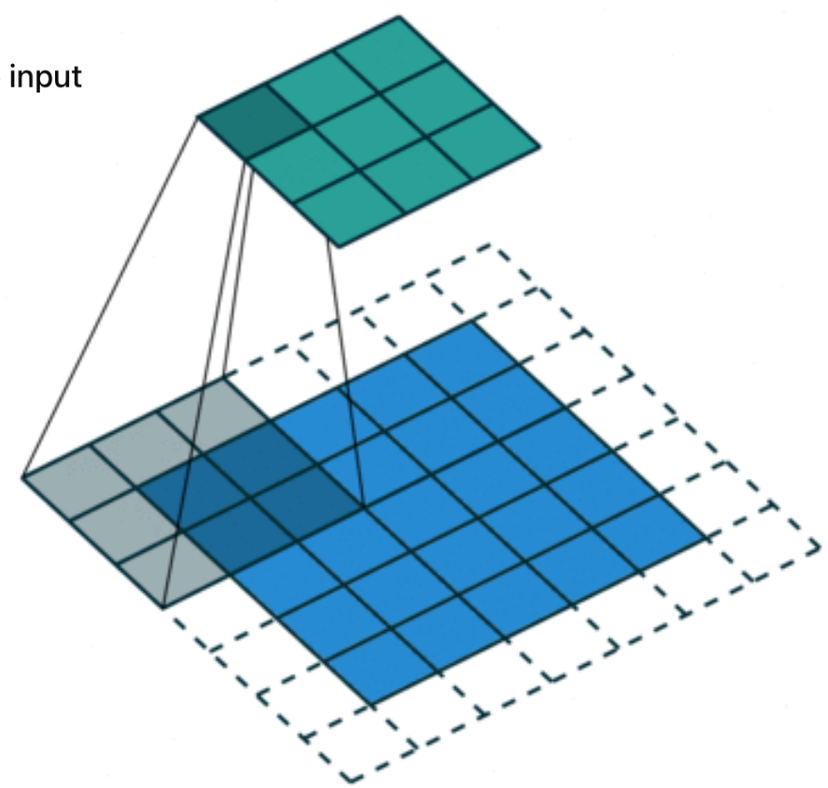
> The number of pixels the filter moves each step

https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html

padding = 0, stride = 2

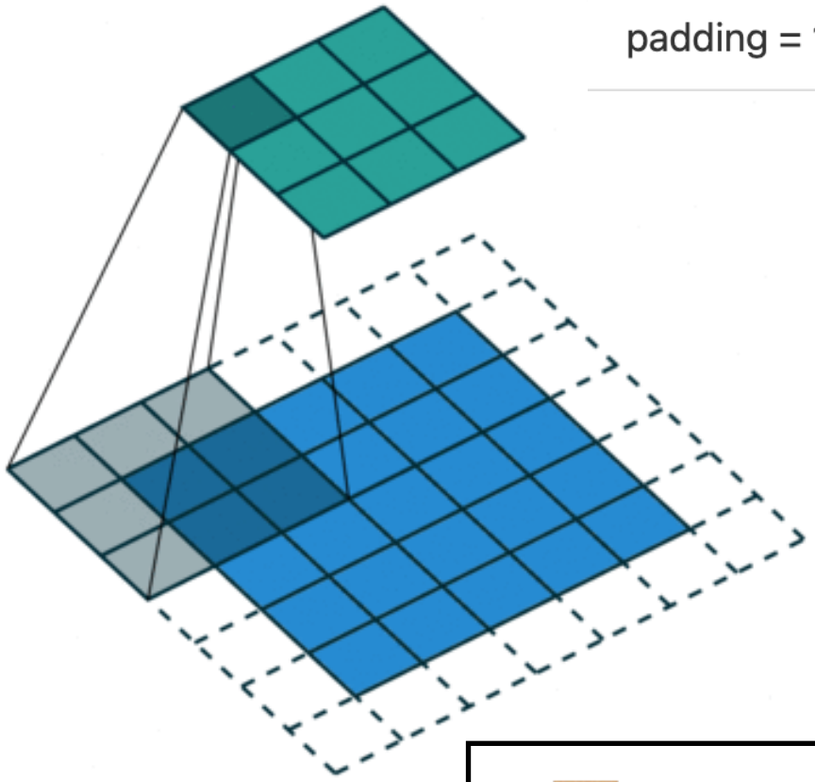


padding = 1, stride = 2

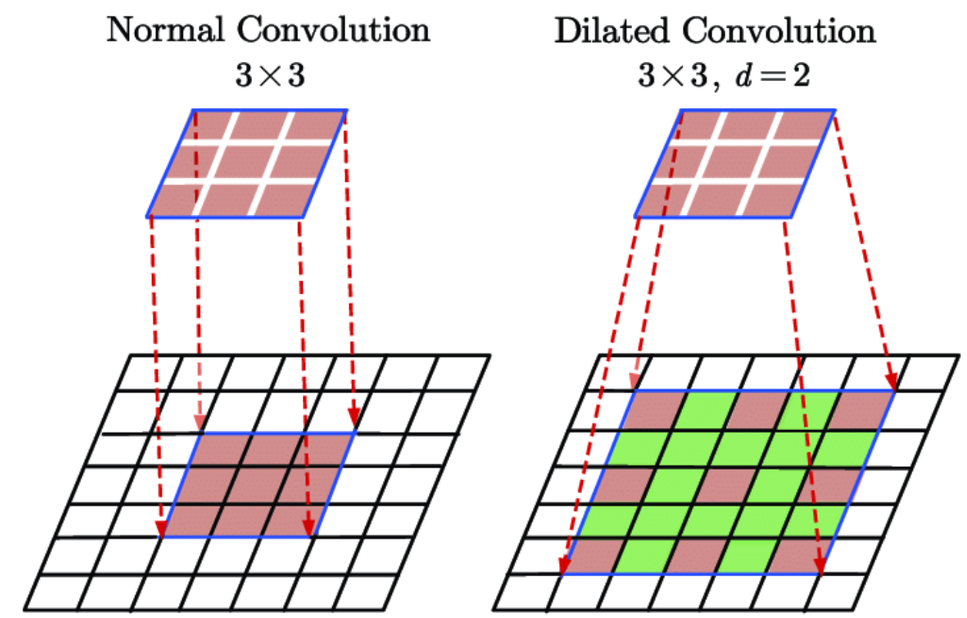


🔹 1. Padding

> Adds pixels (typically zeros) around the border of the input



padding = 1, stride = 2

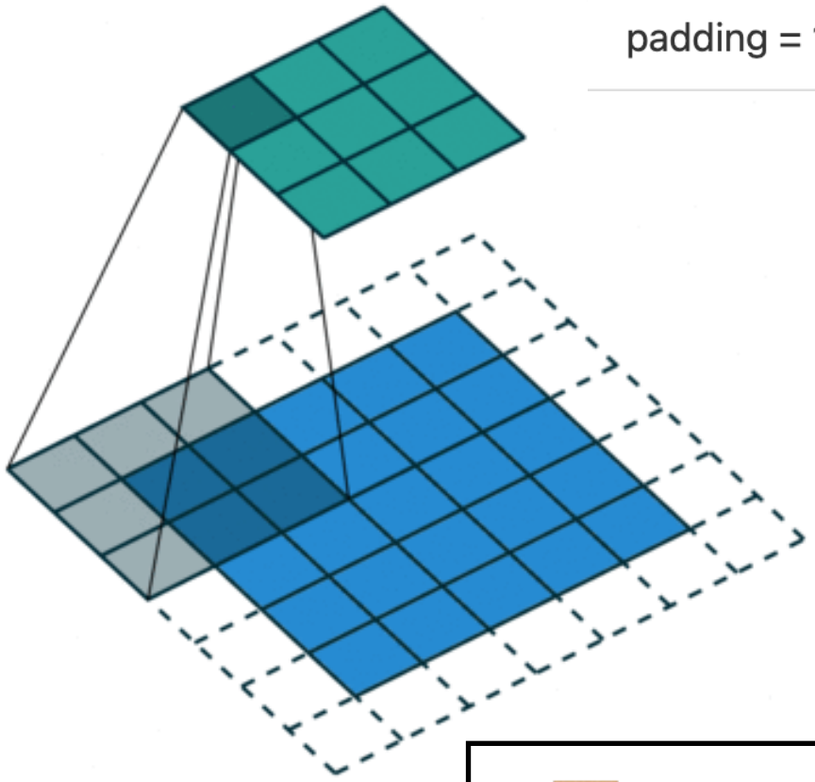


🧮 Formula for Output Size (1D version)

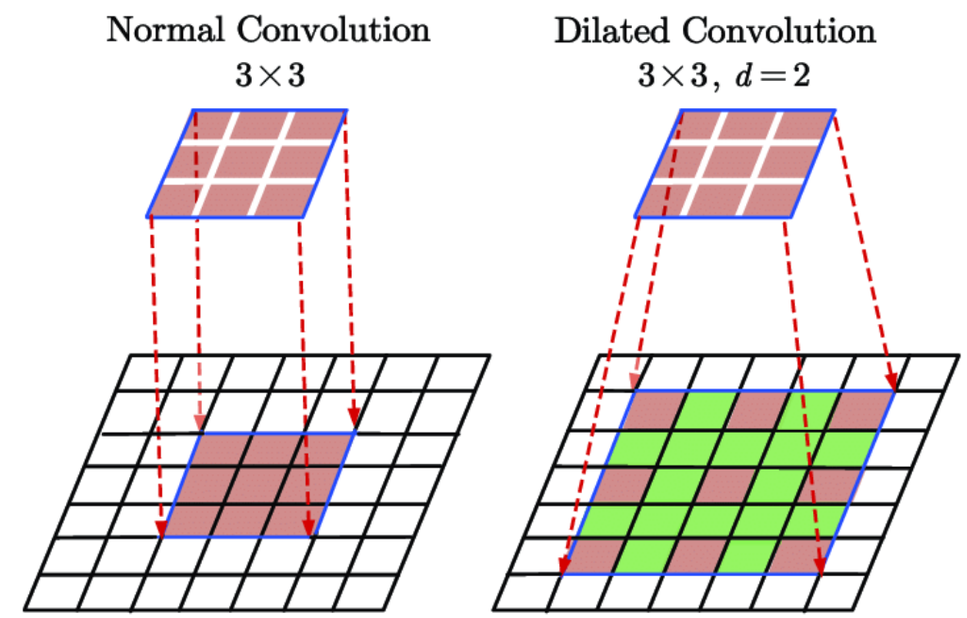
$$\text{Output Size} = \left\lfloor \frac{N + 2P - D \cdot (K - 1) - 1}{S} + 1 \right\rfloor$$

Where:

- N: input size
- P: padding
- K: kernel size
- S: stride
- D: dilation



padding = 1, stride = 2



🧮 Formula for Output Size (1D version)

$$\text{Output Size} = \left\lfloor \frac{N + 2P - D \cdot (K - 1) - 1}{S} + 1 \right\rfloor$$

Where:

- N: input size
- P: padding
- K: kernel size
- S: stride
- D: dilation

Understanding Channels in CNNs

🧠 Key Concepts:

🔷 Channels = Depth of the Data Tensor

- A **grayscale image** has 1 channel → shape `[H, W]`
- An **RGB image** has 3 channels → shape `[3, H, W]`
- In CNNs, inputs are usually shaped `[C_in, H, W]`

Understanding Channels in CNNs

🧠 Key Concepts:

🔷 Channels = Depth of the Data Tensor

- A **grayscale image** has 1 channel → shape `[H, W]`
- An **RGB image** has 3 channels → shape `[3, H, W]`
- In CNNs, inputs are usually shaped `[C_in, H, W]`

When batching images, we use:

- `[N, C_in, H, W]`
  - N: batch size
  - C_{in}: number of **input channels**
  - H, W: height and width

Understanding Channels in CNNs

🧠 Key Concepts:

🔷 Channels = Depth of the Data Tensor

- A **grayscale image** has 1 channel → shape `[H, W]`
- An **RGB image** has 3 channels → shape `[3, H, W]`
- In CNNs, inputs are usually shaped `[C_in, H, W]`

When batching images, we use:

- `[N, C_in, H, W]`
  - N: batch size
  - C_{in}: number of **input channels**
  - H, W: height and width

🔄 Filters Across Channels

- Each filter spans **all input channels**
- For an input with $C_{\text{in}} = 3$, each filter is shaped `[3, K, K]`
- The output at each location is the **sum of convolution outputs across all channels**

Understanding Channels in CNNs

🧠 Key Concepts:

🔷 Channels = Depth of the Data Tensor

- A **grayscale image** has 1 channel → shape `[H, W]`
- An **RGB image** has 3 channels → shape `[3, H, W]`
- In CNNs, inputs are usually shaped `[C_in, H, W]`

When batching images, we use:

- `[N, C_in, H, W]`
  - N: batch size
  - C_{in}: number of **input channels**
  - H, W: height and width

🔄 Filters Across Channels

- Each filter spans **all input channels**
- For an input with $C_{\text{in}} = 3$, each filter is shaped `[3, K, K]`
- The output at each location is the **sum of convolution outputs across all channels**

🧪 Example

- Input: RGB image → shape `[3, 64, 64]`
- First conv layer may use `in_channels=3` and `out_channels=32`
- That means:
  - It takes 3-channel inputs
  - Applies **32 different 3×3×3 filters** to produce **32 output channels**
  - Output shape: `[32, 62, 62]` (assuming no padding)

Understanding `nn.Conv2d` in PyTorch

```python
nn.Conv2d(
    in_channels, out_channels, kernel_size,
    stride=1, padding=0, dilation=1,
    groups=1, bias=True
)
```

Understanding `nn.Conv2d` in PyTorch

```python
nn.Conv2d(
    in_channels, out_channels, kernel_size,
    stride=1, padding=0, dilation=1,
    groups=1, bias=True
)
```

Parameter	Meaning
`in_channels`	Number of input channels (e.g. 3 for RGB)
`out_channels`	Number of filters (→ number of output feature maps)
`kernel_size`	Size of the filters (e.g. `3` or `(3,3)`)
`stride`	How far the kernel moves at each step
`padding`	How many zeros are added around the input
`dilation`	How much spacing between kernel elements
`groups`	For grouped or depthwise conv (advanced)
`bias`	Whether to add a learnable bias per output channel

🧪 Example:

python Copia Modifica

```python
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
```

- Takes RGB input (3 channels)
- Outputs 16 feature maps
- Kernel size is 3×3
- Output shape: same spatial size (because `padding=1`)

🧪 Example:

python Copia Modifica

```python
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
```

- Takes RGB input (3 channels)
- Outputs 16 feature maps
- Kernel size is 3×3
- Output shape: same spatial size (because `padding=1`)

🧮 Output Computation (Mathematical Form)

For each image in batch, for each output channel j:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(j) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(j,k) \star \text{input}(N_i, k)$$

- $\star$ = 2D cross-correlation
- Each output channel combines all input channels using a dedicated filter

CNN Architecture Overview

🔄 General Layer Flow

1. **Input Image**

   → shape: `[C_in, H, W]` (e.g. `[3, 64, 64]`)

2. **Convolution Layer (`nn.Conv2d`)**

   → learns local features via filters

   → outputs feature maps (depth increases)

3. **Activation Function (usually ReLU)**

   → introduces non-linearity

   → allows complex function learning

4. **Pooling Layer (e.g. MaxPool)**

   → reduces spatial size (downsampling)

   → makes representations more abstract and translation-invariant

5. (Repeat steps 2–4 multiple times...)

   → deeper layers learn higher-level features

6. **Flatten**

   → turns 3D feature maps into a 1D vector

7. **Fully Connected Layers (`nn.Linear`)**

   → mix all features together

   → used for final decision-making (classification, regression, etc.)

8. **Output Layer**

   → size = number of classes (e.g. 10 for MNIST, 1000 for ImageNet)

🔷 What Is Max Pooling?

> Max pooling is a **downsampling** operation that reduces the spatial size of feature maps while keeping the most important features.

🔷 What Is Max Pooling?

> Max pooling is a **downsampling** operation that reduces the spatial size of feature maps while keeping the most important features.

🧮 How It Works:

- It slides a small **window** (e.g., 2×2) over the feature map
- For each window, it takes the **maximum value**
- This reduces size by a factor of the **stride**

🔷 What Is Max Pooling?

> Max pooling is a **downsampling** operation that reduces the spatial size of feature maps while keeping the most important features.

🧮 How It Works:

- It slides a small **window** (e.g., 2×2) over the feature map
- For each window, it takes the **maximum value**
- This reduces size by a factor of the **stride**

🧭 Why Use Pooling?

- Reduces **computational cost**
- Increases **receptive field** of subsequent layers
- Adds **translation invariance** (small shifts in input won't change output)
- Helps prevent overfitting

🔷 What Is Max Pooling?

> Max pooling is a **downsampling** operation that reduces the spatial size of feature maps while keeping the most important features.

🧮 How It Works:

- It slides a small **window** (e.g., 2×2) over the feature map
- For each window, it takes the **maximum value**
- This reduces size by a factor of the **stride**

$$\text{Output Size} = \left\lfloor \frac{N - K}{S} + 1 \right\rfloor$$

🧭 Why Use Pooling?

- Reduces **computational cost**
- Increases **receptive field** of subsequent layers
- Adds **translation invariance** (small shifts in input won't change output)
- Helps prevent overfitting

CNN Architecture Overview

🔄 General Layer Flow

1. **Input Image**

   → shape: `[C_in, H, W]` (e.g. `[3, 64, 64]`)

2. **Convolution Layer (`nn.Conv2d`)**

   → learns local features via filters

   → outputs feature maps (depth increases)

3. **Activation Function (usually ReLU)**

   → introduces non-linearity

   → allows complex function learning

4. **Pooling Layer (e.g. MaxPool)**

   → reduces spatial size (downsampling)

   → makes representations more abstract and translation-invariant

5. (Repeat steps 2–4 multiple times...)

   → deeper layers learn higher-level features

6. **Flatten**

   → turns 3D feature maps into a 1D vector

7. **Fully Connected Layers (`nn.Linear`)**

   → mix all features together

   → used for final decision-making (classification, regression, etc.)

8. **Output Layer**

   → size = number of classes (e.g. 10 for MNIST, 1000 for ImageNet)

VAE_Convolution_CELEBA.ipynb

..... then VQ-VAE: Vector Quantised VAE